



Instituto Politécnico Nacional



Escuela Superior de Cómputo
Ingeniería en Sistemas Computacionales
Academia de Sistemas Digitales

Internet de las cosas/Embedded systems

Plan 2020

“Reporte del Proyecto”

Alumnos:

Efrain López Gabriel

Profesor:

Aleman Arce Miguel Ángel

Grupo: 6CM3

Contenido

Introducción.....	2
Objetivo y alcance del sistema.....	2
Arquitectura de comunicación.....	2
Firmware ESP32	3
Responsabilidades principales	3
Portal de configuración y persistencia	3
Medición ultrasónica y cálculo de nivel	4
Control PWM de LED y motor	4
Conexión MQTT y comandos	5
Modo seguro.....	6
Broker Mosquitto y Docker	6
Preparación de Raspberry.....	7
Aplicación Android.....	7
Provisionamiento desde la app.....	8
Conexión MQTT de la app.....	8
Flujo de uso completo.....	9
Pruebas y verificación	9
Resumen del video de prueba	10
Observaciones técnicas.....	11
Conclusiones.....	11
Anexo.....	12

Introducción

AquaControl IoT es un prototipo para monitorear el nivel de agua de un tinaco y controlar una bomba mediante una app Android, una ESP32 y un broker MQTT en Raspberry Pi. El sistema integra medición por sensor ultrasónico, cálculo de nivel, control PWM de indicadores y motor, comunicación MQTT y provisionamiento inicial desde la aplicación móvil.

El proyecto está organizado en cuatro bloques: firmware ESP32, aplicación Android, broker Mosquitto desplegado con Docker Compose y documentación de pruebas/provisionamiento. El reporte desarrolla la arquitectura, el flujo de datos y las decisiones técnicas principales a partir del README, documentos de prueba, scripts de Raspberry y código fuente.

Objetivo y alcance del sistema

El objetivo funcional es medir la distancia del agua respecto al sensor, convertirla en porcentaje de llenado del tinaco, publicar las lecturas por MQTT y permitir que el usuario controle el motor de forma automática o manual desde la app. La Raspberry Pi opera como punto central de mensajería y evita que la app se conecte directamente al microcontrolador durante la operación normal.

Elemento	Función dentro del prototipo	Archivos clave
ESP32	Lee el sensor ultrasónico, calcula nivel, publica distancia/nivel/estado y recibe comandos para el motor.	ESP32/ESP32_Prueba_Sensor_Ultrasonico/Sensor_Ultrasonico_ESP32/Sensor_Ultrasonico_ESP32.ino
Raspberry Pi	Ejecuta Mosquitto en Docker, conserva configuración/datos/logs y funciona como broker MQTT.	compose.yaml; mosquitto/config/mosquitto.conf; Raspberry/setup_raspberry_mqtt.sh
App Android	Provisiona la ESP32 por HTTP, se conecta al broker, visualiza el tinaco y publica comandos.	DashboardActivity.kt; AquaMqttClient.kt; EspProvisioningClient.kt; ProjectConfig.kt
Documentacion	Registra pruebas de ESP32, MQTT, sensor ultrasónico y flujo de provisionamiento.	Docs/provisionamiento_app_esp32_raspberry.md; Docs/prueba_esp32_ultrasonico_mqtt.md

Arquitectura de comunicación

La arquitectura usa MQTT como capa de intercambio entre dispositivos. La ESP32 publica telemetría en topics del prefijo tinaco/ y la app se suscribe a tinaco/# para recibir todos los mensajes relevantes. En sentido contrario, la app publica comandos en tinaco/comando y el firmware los interpreta para activar, detener o reiniciar la configuración.

Tópico	Dirección	Uso principal
tinaco/estado	ESP32 -> App	Estado de conexión, modo seguro, motor o reinicio de configuración.
Tinoco/distancia	ESP32 -> App	Distancia medida por el sensor ultrasónico en centímetros.
Tinoco/nivel	ESP32 -> App	Porcentaje de llenado calculado a partir de la distancia.

Tópico	Dirección	Uso principal
Tinoco/comando	App -> ESP32	Comandos ON, OFF y RESET_CONFIG.
tinaco/#	Broker -> App	suscripción global de la app a los mensajes del sistema.

Durante la operación normal, los tres elementos deben estar en la misma red o tener acceso a la IP del broker. Para el proyecto se definen por defecto el puerto 1883, el usuario MQTT IoTProyecto y la contraseña HOLA. La IP de Raspberry puede variar según red, por lo que la app y el portal de la ESP32 permiten actualizar el host del broker.

Firmware ESP32

Responsabilidades principales

El firmware principal se encuentra en el sketch Sensor_Ultrasonico_ESP32.ino. Integra Wifi, WebServer, Preferences y PubSubClient para resolver tres problemas: configurar la red sin recompilar, medir/controlar el tinaco y mantener una comunicación MQTT estable.

Pin	Función	Detalle de uso
GPIO5	TRIG sensor ultrasonico	Pulso de disparo de 10 microsegundos para iniciar medición.
GPIO18	ECHO sensor ultrasonico	Entrada para medir la duración del eco. Se recomienda adaptar nivel si el sensor entrega 5 V.
GPIO26	LED PWM	Indicador visual proporcional a la distancia medida.
GPIO27	Motor PWM	Salida hacia TIP120 o modulo de potencia para simular bomba.
GPIO0	Boton BOOT	Borra configuración guardada si se mantiene presionado al arrancar.

Portal de configuración y persistencia

Para evitar credenciales fijas en el firmware, la ESP32 crea un punto de acceso temporal cuando no hay configuración guardada o cuando se fuerza el borrado. El portal corre en 192.168.4.1 y expone endpoints HTTP para estado, configuración y reinicio. Los datos se almacenan en la memoria flash con Preferences bajo el namespace aquacontrol.

Fragmento 1. Portal de configuración y valores por defecto. Fuente: Sensor_Ultrasonico_ESP32.ino

```
const char* setup_ap_ssid = "AquaControl-Setup";
const char* setup_ap_password = "aquapi123";
IPAddress setup_ap_ip(192, 168, 4, 1);

const char* mqtt_user_default = "IoTProyecto";
const char* mqtt_password_default = "HOLA";
const int mqtt_port_default = 1883;

void cargarConfiguracion() {
  preferences.begin("aquacontrol", false);
  wifiSsid = preferences.getString("wifi_ssid", "");
  mqttHost = preferences.getString("mqtt_host", "");
  mqttPort = preferences.getInt("mqtt_port", mqtt_port_default);
}
```

La app envía los campos ssid, password, mqtt_host, mqtt_port, mqtt_user y mqtt_password al endpoint /config. Si el firmware no puede conectarse al broker varias veces, abre de nuevo el portal para reconfigurar la IP o las credenciales MQTT sin volver a cargar el programa desde Arduino IDE.

Medición ultrasónica y cálculo de nivel

La medición usa el tiempo de retorno del pulso ultrasónico. El firmware promedia cinco lecturas para suavizar variaciones y calcula el nivel con base en dos distancias de calibración: 30 cm como tinaco vacío y 5 cm como tinaco lleno. El resultado se limita entre 0% y 100% para impedir valores fuera del rango físico esperado.

Fragmento 2. Medición y fórmula de nivel. Fuente: Sensor_Ultrasonico_ESP32.ino

```
float medirDistanciaCm() {
    digitalWrite(PIN_TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(PIN_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);

    unsigned long duracion = pulseIn(PIN_ECHO, HIGH, timeoutUltrasonicoUs);
    if (duracion == 0) return -1.0;
    return duracion * 0.0343 / 2.0;
}

float calcularNivelPorcentaje(float distanciaActual) {
    float nivel = ((distanciaVacio - distanciaActual) /
        (distanciaVacio - distanciaLleno)) * 100.0;
    if (nivel < 0.0) nivel = 0.0;
    else if (nivel > 100.0) nivel = 100.0;
    return nivel;
}
```

Con esta formula, una distancia menor significa mayor llenado. Por ejemplo, si la distancia actual es de 10 cm, el nivel estimado es 80%. El LED y el motor usan proporciones similares, pero con interpretaciones diferentes: el LED aumenta brillo cuando el tinaco se acerca a vacío, y el motor aumenta potencia cuando se activa el control automático.

Control PWM de LED y motor

El LED en GPIO26 funciona como indicador de nivel inverso: apagado cuando el tinaco esta lleno y con brillo máximo cuando esta vacio. El motor en GPIO27 recibe PWM proporcional a la distancia cuando el modo automático esta activo. Como algunos motores no arrancan con PWM bajo, el firmware agrega un pulso inicial de 140/255 durante 300 ms y después ajusta la potencia calculada.

Fragmento 3. Arranque seguro del motor PWM. Fuente: Sensor_Ultrasonico_ESP32.ino

```
void aplicarPotenciaMotorConArranque(int potenciaObjetivo) {
    if (potenciaObjetivo <= 0) {
        aplicarPotenciaMotor(0);
        return;
    }
}
```

```

int potenciaAjustada = potenciaObjetivo;
if (potenciaAjustada < potenciaMotorMinimaGiro) {
    potenciaAjustada = potenciaMotorMinimaGiro;
}

if (potenciaMotorActual == 0 && potenciaAjustada < potenciaMotorArranque) {
    analogWrite(PIN_MOTOR, potenciaMotorArranque);
    delay(duracionPulsoArranqueMotorMs);
}
aplicarPotenciaMotor(potenciaAjustada);
}

```

Conexión MQTT y comandos

La ESP32 construye un clientId dinámico, se autentica con usuario y contraseña, publica online como mensaje retenido y define offline como Last Will and Testament. Al conectarse, se suscribe a tinaco/comando para recibir instrucciones desde la app o desde herramientas de prueba como mosquitto_pub.

Fragmento 4. Conexión MQTT con estado retenido y testamento. Fuente: Sensor_Ultrasonico_ESP32.ino

```

bool conectado = client.connect(
    clientId.c_str(),
    mqttUser.c_str(),
    mqttPassword.c_str(),
    topic_estado,
    0,
    true,
    "offline");

if (conectado) {
    modoSeguroActivo = false;
    intentosMqttFallidos = 0;
    client.publish(topic_estado, "online", true);
    client.subscribe(topic_comando);
}

```

Fragmento 5. Recepción de comandos MQTT. Fuente: Sensor_Ultrasonico_ESP32.ino

```

if (String(topic) == topic_comando) {
    if (mensaje == "ON") {
        controlMotorPorSensorActivo = true;
        modoSeguroActivo = false;
        client.publish(topic_estado, "motor_pwm_auto_on");
    } else if (mensaje == "OFF") {
        controlMotorPorSensorActivo = false;
        aplicarPotenciaMotor(0);
        client.publish(topic_estado, "motor_off");
    } else if (mensaje == "RESET_CONFIG") {
        client.publish(topic_estado, "config_reset_restart");
        borrarConfiguracion();
        ESP.restart();
    }
}
}

```

Modo seguro

El modo seguro desactiva el motor y publica un estado cuando ocurre una condición riesgosa: falla WiFi, falla MQTT, ausencia de lectura del sensor, distancia fuera de rango o tinaco lleno. La decisión de seguridad esta centralizada en `activarModoSeguro()`, lo cual reduce el riesgo de que el motor quede encendido ante errores de comunicación o lectura.

Condición detectada	Respuesta del firmware
Sin lectura ultrasonica	Apaga motor, publica <code>modo_seguro_sensor_sin_lectura</code> y marca distancia como error.
Distancia menor a 2.5 cm o mayor a 30 cm	Apaga motor, conserva publicación de distancia/nivel y reporta fuera de rango.
Wi-Fi desconectado	Apaga motor, intenta reconectar y abre portal si no logra recuperar conexión.
MQTT desconectado	Apaga motor, reintenta cada 5 s y abre portal tras varios fallos.
Tinaco lleno	Apaga motor aunque el usuario este en modo manual.

Broker Mosquitto y Docker

El broker MQTT se ejecuta en la Raspberry Pi mediante Docker Compose. Esta decisión separa la configuración de Mosquitto del sistema operativo y facilita levantar, detener o consultar logs con comandos reproducibles. El contenedor se llama `aquacontrol-mqtt` y expone el puerto 1883 para que ESP32 y Android puedan conectarse por la red local.

Fragmento 6. Servicio Mosquitto en Docker Compose. Fuente: `compose.yaml`

```
services:
  mosquitto:
    image: eclipse-mosquitto:2
    container_name: aquacontrol-mqtt
    restart: unless-stopped
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/config:/mosquitto/config
      - ./mosquitto/data:/mosquitto/data
      - ./mosquitto/log:/mosquitto/log
```

Los volúmenes son importantes: `config` contiene `mosquitto.conf` y `password.txt`; `data` conserva la persistencia del broker; `log` permite separar registros del contenedor. El archivo `password.txt` no se versiona, se crea en la Raspberry y debe quedar con permisos restringidos para el usuario interno 1883 del contenedor.

Fragmento 7. configuración de Mosquitto. Fuente: `mosquitto/config/mosquitto.conf`

```
listener 1883

allow_anonymous false
password_file /mosquitto/config/password.txt
```

```
persistence true
persistence_location /mosquitto/data/

log_dest stdout
```

Directiva	Detalle tecnico
listener 1883	Mosquitto escucha conexiones MQTT TCP en el puerto estándar del proyecto.
allow_anonymous false	Obliga autenticación; la ESP32 y la app usan IoTProyecto/HOLA.
password_file	Apunta al archivo de usuarios creado con mosquitto_passwd dentro del volumen config.
persistence true	Conserva estado persistente del broker en el volumen data.
log_dest stdout	Manda logs a la salida del contenedor para consultarlos con docker compose logs.

Preparación de Raspberry

El script `setup_raspberry_mqtt.sh` automatiza la instalación y puesta en marcha. Configura el hostname, instala Docker, Avahi y mosquitto-clients, verifica que docker compose exista, crea carpetas de Mosquitto, genera el usuario MQTT si no existe y levanta el servicio. Al final publica un mensaje local de prueba en tinaco/estado para validar el broker.

Fragmento 8. Creación de usuario MQTT y arranque del contenedor. Fuente: `Raspberry/setup_raspberry_mqtt.sh`

```
docker run --rm \
-v "$PWD/mosquitto/config:/mosquitto/config" \
eclipse-mosquitto:2 \
mosquitto_passwd -c -b /mosquitto/config/password.txt "$MQTT_USER" "$MQTT_PASSWORD"

sudo chown 1883:1883 mosquitto/config/password.txt
sudo chmod 600 mosquitto/config/password.txt

docker compose up -d
mosquitto_pub -h 127.0.0.1 -u "$MQTT_USER" -P "$MQTT_PASSWORD" \
-t tinaco/estado -m raspberry_mqtt_ready
```

Para operar el broker se usan `docker compose up -d`, `docker compose ps`, `docker compose logs -f mosquitto` y `docker compose down`. Para pruebas MQTT dentro del contenedor, `mosquitto_sub` escucha `tinaco/#` y `mosquitto_pub` publica ON, OFF o RESET_CONFIG en `tinaco/comando`.

Aplicación Android

La app esta escrita en Kotlin con vistas XML. Integra un login local con SQLite, una pantalla de dashboard, un cliente MQTT basado en Eclipse Paho y un cliente HTTP para provisionar la ESP32. `AndroidManifest.xml` habilita `INTERNET` y `usesCleartextTraffic`, necesario para conectarse por HTTP al portal local de la ESP32 y por MQTT TCP al broker de Raspberry.

Módulo	Responsabilidad
MainActivity	y Login local, usuario inicial admin/admin123, creación de usuarios y hash

Módulo	Responsabilidad
AuthDatabaseHelper	SHA-256.
DashboardActivity	Vista principal, provisionamiento, conexión MQTT, control automático/manual y actualización de UI.
AquaMqttClient.kt	conexión Paho, normalización tcp://host:1883, suscripción a tinaco/# y publicación de comandos.
EspProvisioningClient	POST HTTP application/x-www-form-urlencoded al endpoint /config de la ESP32.
TankLevelView	Vista personalizada para representar visualmente el porcentaje de llenado.

Provisionamiento desde la app

En el primer arranque la ESP32 abre AquaControl-Setup. El teléfono se conecta a esa red, la app envía los datos de la red real y del broker MQTT al portal 192.168.4.1/config, y el microcontrolador guarda la configuración antes de reiniciarse. Después, tanto el teléfono como la ESP32 vuelven a la red normal y se comunican por Mosquitto.

Fragmento 9. Envío HTTP de configuración al portal ESP32. Fuente: EspProvisioningClient.kt

```
val body = formBody(
    "ssid" to request.wifiSsid,
    "password" to request.wifiPassword,
    "mqtt_host" to request.mqttHost,
    "mqtt_port" to request.mqttPort.toString(),
    "mqtt_user" to request.mqttUser,
    "mqtt_password" to request.mqttPassword,
)

OutputStreamWriter(connection.outputStream, Charsets.UTF_8).use { writer ->
    writer.write(body)
}
```

Conexión MQTT de la app

AquaMqttClient normaliza el host ingresado por el usuario. Si el texto no incluye protocolo, antepone tcp://; si no incluye puerto, agrega 1883. La conexión se crea con clean session, reconexion automatica, timeout de 8 segundos y keep alive de 20 segundos. Tras conectar, la app se suscribe a tinaco/# con QoS 1.

Fragmento 10. Suscripción y publicación desde Android. Fuente: AquaMqttClient.kt

```
mqttClient.subscribe(
    ProjectConfig.TOPIC_ALL,
    1,
    null,
    mqttActionListener(
        onSuccess = { listener.onSubscribed(ProjectConfig.TOPIC_ALL) },
        onFailure = { exception ->
            listener.onError("Conecto, pero fallo la suscripcion: ${exception.safeMessage}")
        },
    ),
)
```

```

val message = MqttMessage(command.toByteArray(Charsets.UTF_8)).apply {
    qos = 1
    isRetained = false
}
activeClient.publish(
    ProjectConfig.TOPIC_COMMAND,
    message,
    null,
    mqttActionListener(
        onSuccess = { listener.onCommandPublished(command) },
        onFailure = { exception ->
            listener.onError("No se pudo publicar $command: ${exception.safeMessage}")
        },
    ),
)
)

```

DashboardActivity interpreta tinaco/distancia, tinaco/nivel y tinaco/estado para actualizar textos y la vista TankLevelView. En modo automático, si el nivel baja al umbral configurado, por defecto 30%, publica ON. Cuando el nivel llega a 100%, publica OFF por seguridad. En modo manual el usuario puede encender o apagar, pero el llenado completo también fuerza apagado.

Flujo de uso completo

Paso	Acción	Resultado esperado
1	Levantar Mosquitto en Raspberry con docker compose up -d.	Contenedor aquacontrol-mqtt activo en puerto 1883.
2	Energizar la ESP32 con el firmware principal.	Si no hay configuración, aparece AquaControl-Setup en 192.168.4.1.
3	Conectar el celular a AquaControl-Setup y enviar configuración desde la app.	La ESP32 guarda WiFi/broker y se reinicia.
4	Volver el celular a la red normal y pulsar Conectar.	La app se suscribe a tinaco/# y muestra MQTT listo.
5	Observar nivel/distancia y seleccionar modo automático o manual.	La ESP32 publica telemetría y responde a ON/OFF.
6	Si cambia la IP del broker, usar RESET_CONFIG o botón BOOT.	La ESP32 regresa al portal de configuración.

Pruebas y verificación

La documentación del proyecto registra primero una prueba MQTT básica entre ESP32 y Raspberry, y después la integración con sensor ultrasónico, LED y motor. La prueba básica valida comunicacion bidireccional: ESP32 publica en tinaco/estado y tinaco/nivel, mientras Raspberry envía ON/OFF a tinaco/comando.

Fragmento 11. Escucha de mensajes MQTT desde Raspberry. Fuente: README.md

```

docker exec -it aquacontrol-mqtt mosquitto_sub \
    -h localhost \
    -u IoTProyecto \

```

```
-P HOLA \
-t 'tinaco/#' \
-v
```

Validación	Criterio de éxito
Broker	docker compose ps muestra aquacontrol-mqtt activo y logs sin errores de autenticación.
ESP32	Monitor Serial muestra WiFi conectado, MQTT conectado y suscripción a tinaco/comando.
Sensor	Distancias estables dentro de 2.5 cm a 30 cm; error controlado si no hay eco.
MQTT	Raspberry recibe tinaco/distancia, tinaco/nivel y tinaco/estado con mosquitto_sub.
Control	ON activa control automático del motor; OFF apaga; RESET_CONFIG reinicia provisionamiento.
App	Dashboard muestra MQTT listo, nivel, distancia, estado ESP32 y ultimo mensaje recibido.

Resumen del video de prueba

El archivo prueba.mp4 documenta una prueba integral del prototipo. La grabación tiene una duración aproximada de 2 minutos con 11 segundos, incluye video de 848 x 478 pixeles y conserva audio, lo cual permite observar tanto la respuesta visual de la app como escuchar la activación del motor. La secuencia inicia con el arranque del contenedor Docker con Mosquitto, continua con la vinculación de la ESP32 con la aplicación Android y el broker MQTT, y termina con la validación dinámica de lecturas al mover la protoboard con el sensor ultrasónico.

Durante la demostración se desplaza físicamente el sensor para modificar la distancia medida. Como resultado, la app actualiza la representación gráfica del tinaco, lo que evidencia que los mensajes publicados por la ESP32 en tinaco/distancia y tinaco/nivel son recibidos e interpretados correctamente.

Además, el audio permite identificar el momento en que se activa el motor, confirmando que la parte de actuación también responde al flujo de control.

Momento del video	Evidencia observada	Validación del sistema
Arranque del broker	La grabación inicia con el levantamiento del servicio Docker que ejecuta Mosquitto.	Confirma que el broker aquacontrol-mqtt queda disponible como punto central MQTT.
Enlace ESP32 - app - MQTT	después se observa la vinculación de la ESP32 con la aplicación y el broker MQTT.	Valida el flujo de provisionamiento, conexión a red, autenticación MQTT y suscripción a tinaco/#.
Movimiento del sensor	Se desplaza la protoboard con el sensor ultrasónico para cambiar la distancia medida.	Demuestra que la ESP32 calcula nuevos valores y publica cambios de distancia/nivel.
Respuesta visual en la app	La representación del tinaco en la app cambia conforme varían las lecturas del	Comprueba la recepción de tinaco/nivel y la actualización de la interfaz en tiempo

Momento del video	Evidencia observada	Validación del sistema
	sensor.	real.
Activación del motor	En el audio se escucha cuando el motor se activa durante la prueba.	Evidencia que el comando/control PWM llega al actuador y que el sistema responde físicamente.

En conjunto, el video funciona como evidencia practica de extremo a extremo: infraestructura MQTT activa, comunicacion entre dispositivos, actualización de la interfaz móvil y respuesta física del actuador.

Observaciones técnicas

El diseño actual es adecuado para prototipo local y pruebas de laboratorio. La autenticación MQTT evita conexiones anónimas, pero las credenciales aparecen como valores por defecto en firmware y app; para un despliegue real convendría rotarlas, evitar contraseñas fijas y considerar TLS o una red aislada. El control del motor también debe validarse eléctricamente: el GPIO no alimenta el motor directamente, por lo que se requiere etapa de potencia, diodo de protección y tierra común entre fuente y ESP32.

El uso de Docker Compose simplifica el despliegue de Mosquitto en Raspberry y facilita recuperar el servicio. La persistencia activada permite conservar estado en mosquitto/data, y log_dest stdout alinea los mensajes con docker compose logs. En el firmware, la apertura automática del portal tras fallos MQTT es una mejora importante porque resuelve cambios de IP sin reprogramar la placa.

Conclusiones

AquaControl IoT integra correctamente una arquitectura IoT local con ESP32, Android y Raspberry Pi. El firmware resuelve medición, cálculo de nivel, actuación PWM, provisionamiento y seguridad básica. La app aporta la interfaz de usuario, el alta de configuración y el control automático/manual. Mosquitto en Docker funciona como broker central y permite pruebas reproducibles con herramientas de línea de comandos.

El punto mas importante del proyecto es la separación de responsabilidades: la ESP32 se especializa en sensor y actuadores, la app en experiencia de usuario y reglas de control, y Raspberry en mensajería MQTT persistente. Esta división deja una base clara para evolucionar el prototipo con autenticación más robusta, telemetría histórica, alertas y mejoras de hardware.

Anexo

```
eefrain@2806-106e-0027-164d-d8d9-ade4-68f1-e088-~/Escriptorio/IoT-Proyecto$ docker compose up -d
[*] up 2/2
✓ Network iot-proyecto.default Created
✓ Container aquacontrol-mqtt Started
eefrain@2806-106e-0027-164d-d8d9-ade4-68f1-e088-~/Escriptorio/IoT-Proyecto$ docker exec -it aquacontrol-mqtt mosquito_sub -h localhost -u IoTProyecto -P HOLA -t 'tinaco/#' -v
tinaco/estado online
tinaco/distancia 4.28
tinaco/nivel 100.0
tinaco/distancia 5.30
tinaco/nivel 98.8
tinaco/distancia 6.43
tinaco/nivel 94.3
tinaco/distancia 8.94
tinaco/nivel 84.3
tinaco/distancia 10.56
tinaco/nivel 77.8
tinaco/distancia 12.56
tinaco/nivel 69.8
tinaco/distancia 16.59
tinaco/nivel 53.6
tinaco/distancia 20.37
tinaco/nivel 38.5
tinaco/distancia 20.23
tinaco/nivel 39.1
tinaco/distancia 21.46
tinaco/nivel 34.1
tinaco/distancia 22.33
tinaco/nivel 30.7
tinaco/distancia 22.97
tinaco/nivel 28.1
tinaco/comando ON
tinaco/estado motor_pwm_auto_on
tinaco/distancia 23.08
tinaco/nivel 27.7
tinaco/distancia 20.46
tinaco/nivel 30.46
tinaco/estado motor_off
tinaco/distancia 4.57
tinaco/nivel 100.0
tinaco/distancia 4.56
tinaco/nivel 100.0
tinaco/distancia 4.56
tinaco/nivel 100.0
tinaco/distancia 4.56
tinaco/nivel 100.0
tinaco/distancia 4.56
tinaco/nivel 100.0
tinaco/distancia 4.56
tinaco/nivel 100.0
tinaco/distancia 4.57
tinaco/nivel 100.0
tinaco/distancia 4.58
tinaco/nivel 100.0
tinaco/distancia 4.56
tinaco/nivel 100.0
tinaco/distancia 4.56
tinaco/nivel 100.0
tinaco/distancia 4.68
tinaco/nivel 100.0
tinaco/distancia 4.78
tinaco/nivel 100.0
tinaco/distancia 4.84
tinaco/nivel 100.0
tinaco/distancia 4.77
tinaco/nivel 100.0
tinaco/distancia 4.57
tinaco/nivel 100.0
tinaco/distancia 4.59
tinaco/nivel 100.0
tinaco/distancia 4.60
tinaco/nivel 100.0
tinaco/distancia 4.60
tinaco/nivel 100.0
tinaco/distancia 4.59
tinaco/nivel 100.0
tinaco/distancia 4.66
tinaco/nivel 100.0
tinaco/distancia 4.65
tinaco/nivel 100.0
tinaco/distancia 4.76
tinaco/nivel 100.0
tinaco/distancia 4.65
tinaco/nivel 100.0
tinaco/distancia 4.66
tinaco/nivel 100.0
tinaco/distancia 4.65
tinaco/nivel 100.0
tinaco/distancia 4.65
tinaco/nivel 100.0
tinaco/distancia 4.68
tinaco/nivel 100.0
^C
eefrain@2806-106e-0027-164d-d8d9-ade4-68f1-e088-~/Escriptorio/IoT-Proyecto$ docker compose down
[*] down 2/2
✓ Container aquacontrol-mqtt Removed
✓ Network iot-proyecto.default Removed
eefrain@2806-106e-0027-164d-d8d9-ade4-68f1-e088-~/Escriptorio/IoT-Proyecto$
```

10:17

Conexion MQTT

Bluetooth, Cellular, Wi-Fi, 42%

Host/IP Raspberry o broker

192.168.1.69

Red WiFi de trabajo

INFINITUM5516

Conectar

Desconectar

Configuracion inicial ESP32

Primero conecta el celular a AquaControl-Setup. Luego envia el WiFi y el broker que usara la ESP32.

Host del portal ESP32

192.168.4.1

WiFi al que se conectara el ESP32

INFINITUM5516

.....

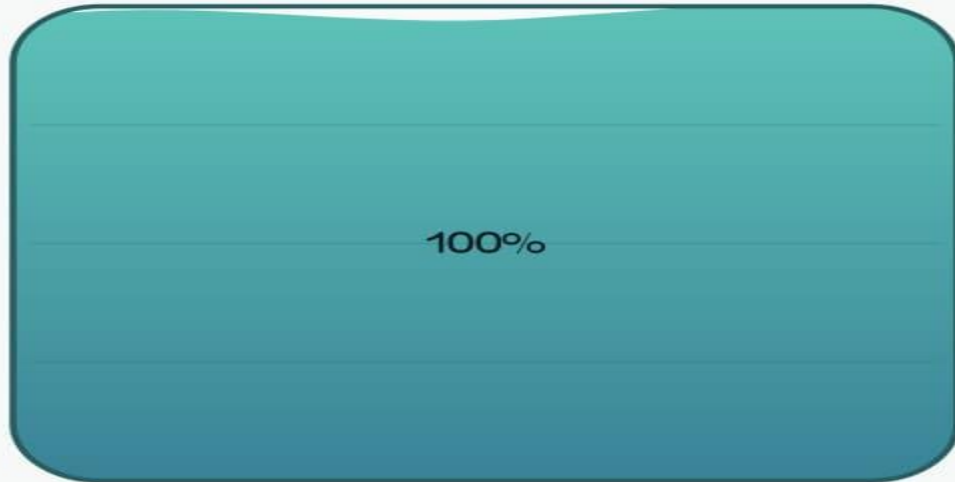
Credenciales MQTT

IoTProyecto

....

Enviar configuracion al ESP32

Borrar WiFi guardado del ESP32



100%

100%

Distancia: 4.6 cm

ESP32: motor_off

Modo de control

- ☒ Automatico: encender al 30% y apagar al llenarse
- ☐ Manual: encender cuando yo quiera, apagar al llenarse

Umbral automatico (%)

30

Aplicar modo automatico

Motor ON

Motor OFF

Tinaco lleno: motor apagado por seguridad

Ultimo MQTT: Conexion MQTT perdida: Se ha perdido la conexión